自建 remote MCP server

踩坑手册

Claude / ChatGPT 通用

按症状找

403 公网 403、本机 200，body 是 `Invalid Host` 第二节 →

500 该 401 的地方返回 500 第三节 →

404 `Couldn't register with … sign-in service` 第一节 →

面向：所有基于 Streamable HTTP 的 remote MCP server。文中以 Claude、ChatGPT 网页端（含桌面 / 手机 App）、Claude Code、Codex 等客户端为例进行说明与验证。示例技术栈为 Node.js + @modelcontextprotocol/sdk。内容来自自己部署远程 MCP 服务时实际踩到的坑，基于 SDK 1.29 写成，并于 2026-10 对照 1.32.1 复核。

这些坑的共同特征：**症状指向的位置和真正的原因不在一处**。所以每条都先写现象。

目 录

先分清两类客户端 3

先决条件 3

零 先用 SDK 自带的零件，别从零设计 4

一 authless 服务：三个探测端点 7

二 DNS rebinding 保护：那个 403 不是网关拦的 8

三 `requireBearerAuth` 的三个硬约束 8

四 自建 OAuth 授权服务器 10

五 测试这类服务时自己会踩的坑 11

六 从 Windows PowerShell 经 SSH 部署 12

七 排错速查 13

参考 14

先分清两类客户端

这是理解后面所有坑的前提：

- **网页端**（Claude 官方端、ChatGPT）：是平台的服务器替你去连 MCP 服务，请求从 Anthropic / OpenAI 的机房发出，不是从你的电脑。所以服务必须公网可达，而且两家都基本只认 OAuth。
- **本机客户端**（Claude Code、Codex 等）：从你自己的电脑发请求。

好消息是，**做好 OAuth 之后，本机客户端不用另外配**：

1. **首选：在网页端授权一次**。在 claude.ai 上用 OAuth 连好的连接器，登录同一账号的 Claude Code 和 Claude 桌面版会直接带过来，不用再配置，也不用再登录。
2. **客户端自己走 OAuth**。不登录账号、或者用别家客户端时，客户端会自己发起 OAuth，回跳到 `http://localhost:<随机端口>/callback`。你的服务得放行这种本机回跳才行（见第四节）。
3. **静态 Bearer 请求头**。前两条都走不通，或者客户端根本不支持 OAuth（比如一些自建 App），就在配置里填 `Authorization: Bearer ...`。这条路 SSH 隧道、内网地址也能用。

先决条件

- **给网页端用的服务必须公网可达**。仅私网可达、位于内网或被防火墙阻断的地址直接添加是连不上的。要在防火墙放行的话，两家都公布了出口 IP（Anthropic 是 `160.79.104.0/21`，OpenAI 见其 IP 地址文档）。Claude 另外提供了给内网服务用的 MCP tunnels。
- **令牌绝不能放在 URL 查询串里**（`?token=` 这种），MCP 授权规范明确禁止。网上有些“测试用”教程教你这么绕过 OAuth，token 会留在各种日志里，别学。

各端支持的认证方式（2026-10，以官方文档为准）：

方式	Claude 网页端	ChatGPT	本机客户端
OAuth 2.x + PKCE（CIMD / DCR 自动注册）	默认可用	默认可用（且是唯一的鉴权方式）	可用（见上面第 1、2 条）
静态请求头（API key / Bearer token）	Beta，仅部分组织可用	不支持	直接填
无认证（authless）	可用	可用	可用

个人开发者也可以添加自己的 MCP 服务，**读和写工具都能用**，OpenAI 开发者文档原话是 “ChatGPT supports both read and write tools from your server.”新文档没有按订阅档位写限制，具体能力按账号开通。OpenAI 帮助中心有一篇旧文章说个人档位“只能读”，跟新文档对不上，以新文档为准。

另外，ChatGPT 会根据工具声明决定要不要先让你确认。纯读取的工具记得在定义里加 `annotations: { readOnlyHint: true }`，不然可能被当成会改数据的工具；会造成难以撤销后果的工具，标上 `destructiveHint: true`。

所以只要服务里有私人数据、又想在网页端用，**自建 OAuth 基本绕不开**。好在 SDK 把大部分活干了，第零节和第四节讲怎么做。

第 零 节

先用 SDK 自带的零件，别从零设计

SDK 已经把鉴权协议和 HTTP 层的大部分活干了。新建服务时按这几条来，能绕开下面一到五节里大部分坑：

1. **OAuth 端点用** `mcpAuthRouter`，它会提供 `/authorize`、`/token`、`/register`、`/revoke` 和两个 `.well-known` 发现端点。你只需要实现一个 `OAuthServerProvider`（第四节列了安全要点）。
2. **校验令牌用** `requireBearerAuth`，但要满足第三节的三个硬约束，不然该 401 的地方会变 500。

3. Express 应用用 `createMcpExpressApp`，并显式传 `allowedHosts`（见第二节）。
4. **`/mcp` 的 StreamableHTTP 会话处理照 SDK 示例写一份，抽成公共函数。**如果你有不止一台服务，让它们共用这一份，不要每台各抄一遍。

鉴权的组装大致是这样（主力走 OAuth；没配 OAuth 时退回只认静态 token，给本机客户端用）：

```
const {
  mcpAuthRouter,
  getOAuthProtectedResourceMetadataUrl
} = require('@modelcontextprotocol/sdk/server/auth/router.js')
const {
  requireBearerAuth
} = require('@modelcontextprotocol/sdk/server/auth/middleware/bearerAuth.js')

let authorize
if (oauth) {
  const issuerUrl = new URL(oauth.issuer)
  const resourceServerUrl = new URL('/mcp', issuerUrl)
  app.use(
    mcpAuthRouter({
      provider: oauth,
      issuerUrl,
      resourceServerUrl,
      resourceName: '服务名'
    })
  )
  authorize = requireBearerAuth({
    verifier: oauth,
    resourceMetadataUrl: getOAuthProtectedResourceMetadataUrl(resourceServerUrl)
  })
} else {
  // 隧道模式：只给本机客户端用的静态 token
  authorize = (req, res, next) => {
    if (!accessToken || req.headers.authorization !== `Bearer ${accessToken}`) {
      return res.status(401).json({ error: 'Unauthorized' })
    }
    next()
  }
}
app.all('/mcp', authorize, handleMcp) // handleMcp 就是上面第 4 条抽出来的会话处理
```

两条路怎么选：对外地址（`PUBLIC_URL`）和同意页口令**都配了**才启用 OAuth，缺任何一项就退回只认静态 token 的隧道模式——不要做成“半开的门”。静态 token 至少 32 位随机值，启动时就检查，别等运行时才发现。

新建服务的检查清单：

- [ ] 用了 `mcpAuthRouter` / `requireBearerAuth` / `createMcpExpressApp`，没有手写协议端点
- [ ] 对外服务设了 `allowedHosts`（否则公网 403，见第二节）
- [ ] 静态 token ⩾32 位且启动时检查
- [ ] 有 `/health` 端点
- [ ] 多台服务的端口 / token / 配置文件 / systemd 服务各自独立，不共用凭据
- [ ] OAuth 注册测试同时覆盖你要支持的每个客户端域名（Claude 是 `claude.ai` 和 `claude.com`，ChatGPT 是 `chatgpt.com`），并确认会拒绝 http loopback、后缀伪装（`claude.ai.evil.com`）和陌生域名
- [ ] 写了走真实 HTTP 的集成测试（见第五节）

多台服务共用代码时，别只上传一个文件

几台服务共用一份公共模块（比如上面抽出来的会话处理）时，本地代码往往比线上新。`node --check server.js` 只检查这一个文件的语法，**不会**检查它 `require()` 进来的模块是不是导出了它要用的函数。所以“这次只改了一行”不等于“只传一个文件就行”：

1. 先看线上这台服务 `require()` 了哪些公共模块、这些模块现在导出了什么；
2. 列出这次改动真正依赖的全部文件，一起上传；
3. 覆盖公共模块会影响之后重启的**所有**引用它的服务，先确认兼容，再建好整批回退点；
4. 一次只重启一台，逐台验证 health、OAuth 元数据、匿名请求是否 401、客户端能否正常调用。

第一节

authless 服务：三个探测端点

现象 | 在 Claude 添加自定义连接器时报 `Couldn't register with XXX's sign-in service`（其他客户端的报错文字不同，但原因一样）。

原因 | 按 MCP 授权规范，客户端在连接前会先探测这三个地址，判断服务器要不要 OAuth：

1. `GET /.well-known/oauth-protected-resource`
2. `GET /.well-known/oauth-authorization-server`
3. `POST /register`

只要响应不标准（纯文本、HTML 错误页、200 加奇怪 body），客户端就会推断「这个服务器需要 OAuth 但没配好」，转而走动态注册流程然后失败。

修法 | 让这三个端点返回**标准的 404 + JSON**：

```
// 必须排在所有鉴权检查之前，否则 401 又会让客户端以为要走 OAuth
if (
  req.url === '/.well-known/oauth-protected-resource' ||
  req.url === '/.well-known/oauth-authorization-server' ||
  req.url === '/register'
) {
  res.writeHead(404, { 'Content-Type': 'application/json' })
  res.end(JSON.stringify({ error: 'not_found' }))
  return
}
```

四个要点缺一不可：状态码是 **404**、Content-Type 是 **application/json**、body 是**合法 JSON**、位置在**鉴权之前**。

> 用 `mcpAuthRouter` 做 OAuth 的服务不需要这一段——那些端点由 SDK 提供真实内容。

第 二 节

DNS rebinding 保护：那个 403 不是网关拦的

现象 | 本机 `curl 127.0.0.1:PORT/health` 返回 200，公网走域名访问同一个路径返回 403。很容易被当成 Cloudflare Access、WAF 或反向代理的问题，然后往错误的方向查很久。

辨认 | 看响应体和响应头。

```
X-Powered-By: Express
{"jsonrpc":"2.0","error":{"code":-32000,"message":"Invalid Host:
your.domain"},"id":null}
```

带 `X-Powered-By: Express` 和 JSON-RPC 结构，就说明**是源站自己拒绝的**，跟网关无关。Cloudflare Access 的拒绝页是 HTML。

原因 | `createMcpExpressApp()` 在绑定 localhost 时会自动开启 DNS rebinding 保护，只接受 `127.0.0.1` / `localhost` / `::1` 的 Host 头。经域名进来的请求全被 403。

修法 | 把对外域名显式加进白名单。

```
createMcpExpressApp({
  allowedHosts: ['127.0.0.1', 'localhost', '[::1]', 'your.domain']
})
```

校验不看端口（内部用 URL API 取 hostname），所以只填主机名，不要带端口。不要为了图省事绑到 `0.0.0.0` 把保护整个关掉——它挡的是真实的攻击面。

第 三 节

`requireBearerAuth` 的三个硬约束

这三条都会让**本该 401 的响应变成 500**，从外面看完全一样，得读 SDK 源码才分得清。

3.1 AuthInfo 必须带 `expiresAt`

中间件会检查 `verifyAccessToken` 返回的 `expiresAt` 是不是数字，不是就当无效令牌处理。所以返回的对象**必须**有 `expiresAt`（秒级时间戳）。如果你同时支持一个不会过期的静态 token，也得给它编一个滚动的过期时间，否则那条路会被判成无效令牌——**看起来就像认证配置错了**。

3.2 错误必须是 `OAuthError` 的子类

中间件用 `instanceof` 分派状态码。抛普通 `Error` 会落到兜底分支，变成 500。

场景	该抛的错误	抛错了的后果
`verifyAccessToken` 校验失败	`InvalidTokenError`	401 → 500
`registerClient` 拒绝注册	`InvalidClientMetadataError`	400 → 500
作用域不足	`InsufficientScopeError`	403 → 500

都从 `@modelcontextprotocol/sdk/server/auth/errors.js` 导入。

3.3 会进响应头的错误消息必须是纯 ASCII

中间件把 message 直接拼进 `WWW-Authenticate`：

```
;`Bearer error="${errorCode}", error_description="${message}"`
```

HTTP 头不允许非 ASCII 字符。写中文的错误消息 → Node 抛 `TypeError [ERR_INVALID_CHAR]` → **401 又退化成 500**。

`InvalidTokenError` 的消息一律用英文。进 JSON body 的错误（token / register 端点）可以用中文，但统一用 ASCII 更省心。

第四节

自建 OAuth 授权服务器

SDK 自带脚手架，协议部分不用自己写：

```
server/auth/router.js          mcpAuthRouter: /authorize /token /register /revoke +
两个 .well-known
server/auth/handlers/          各端点实现
server/auth/middleware/        requireBearerAuth
server/auth/provider.js        要实现的接口
```

实现 `OAuthServerProvider` 即可。SDK 的 `mcpAuthRouter` 已经在元数据里声明了 `code_challenge_methods_supported: ["S256"]` 和 public client（`none`），这是 Claude 和 ChatGPT 都要求的；它没声明 CIMD，两家会自动退回用动态注册（DCR），不影响使用。

几个安全要点：

- **动态注册（DCR）是对所有人开放的**，这是规范要求，拦不住。所以真正的门是 `/authorize` 的同意页——它必须有一道只有你本人知道的凭据（口令 / PIN），否则任何人注册一个客户端就能换到令牌。口令最好再加上“错几次就锁定”。
- **必须校验 `redirect_uri`**：限定 https + 受信域名白名单。判断域名时要用 `hostname === host || hostname.endsWith("." + host)`，直接 `includes` 会被 `claude.ai.evil.com` 这种后缀伪装骗过去。
- **同意页要显示客户端名和回跳地址**，让人能在授权前看出异常。
- **authorization code 一次性、短期**，并绑定 client_id、redirect_uri 和 code_challenge；换取令牌时逐项比对。
- **refresh token 轮换**：用掉即作废。
- **令牌落盘只存摘要**（SHA-256），不存明文。
- 回跳地址必须**精确匹配**，差一个字符最后一步就失败：
  - Claude 网页端：`https://claude.ai/api/mcp/auth_callback`

    - ChatGPT：形如 `https://chatgpt.com/connector/oauth/{callback_id}`，每个连接不一样，以 ChatGPT 管理页上显示的为准。白名单按域名放行 `chatgpt.com` 就不用逐个登记。
- Claude 添加连接器时 OAuth Client ID 一般可留空（自动注册）。如果你想填，就在服务端预先注册一个 client 把 id 给它；`token_endpoint_auth_method: "none"` 的 public client 没有 secret，Secret 栏留空即可。
- **受信域名白名单要按「这台服务实际给哪几端用」来配，不要照抄另一台。**我有一台服务只给 ChatGPT 用，白名单只写了 `chatgpt.com`；另一台照抄之后，Claude 在注册这一步就被 400，看起来像客户端 bug。Claude 要 `claude.ai` 和 `claude.com` 两个。
- **本机客户端自己走 OAuth 时，回跳是 `http://localhost:<随机端口>/callback`**（比如在终端里 `claude mcp add` 后登录），是 http loopback，过不了「https + 受信域名」这道校验。Claude 官方文档建议授权服务器对 `localhost` 和 `127.0.0.1` 做不看端口的匹配来放行它；代价是本机任何进程都能来注册，不过拿令牌仍要过同意页的口令。我选择不放：终端用继承来的账号连接器，或者在本机配置里填静态 Bearer，就够了（我有一台服务就是踩了这个坑之后改成静态 Bearer 的）。
- **换域名要连 OAuth issuer 一起换。**issuer 通常就是 `PUBLIC_URL`，而它往往同时是 `allowedHosts` 的来源；只改反向代理不改配置，不是 issuer 对不上就是 403。换完已授权的客户端要重新授权。

第 五 节

测试这类服务时自己会踩的坑

前四类坑里有几个，本地单元测试一**个都抓不到**，因为它们直接调 provider，绕过了中间件和真实的 HTTP 层。

- **必须写走真实 HTTP 的集成测试：**`app.listen(0)` 起随机端口，然后发真请求。「测试覆盖到那一行」不等于「测试验证了那个行为」。
- **fetch 的 header 值必须是 ASCII。**用中文当假 token（`Bearer 瞎编的`）会让 undici 在**构造请求时**就抛错，请求根本没发出去，你以为测了其实没测。

- **fetch 不能覆盖 `Host`**（forbidden header name）。要验证 Host 白名单，必须用 `node:http` 直接发请求。
- 断言要覆盖**完整的请求形状**，不只是自己关心的那个片段。我有过一个测试传了 `memoryType: "diary"` 却只断言了另一个字段，于是漏掉了「过滤值少了前缀」这个让整个功能失效的 bug。

第六节

从 Windows PowerShell 经 SSH 部署

线上目录不一定是 Git 仓库，部署前先从实际的 systemd 服务配置核对 `WorkingDirectory`、`ExecStart` 和依赖路径，不要凭本地模板猜。

安全顺序固定为：

1. SSH 只读检查服务、监听端口、服务配置和目标文件，确认这次要改的精确范围；
2. 在服务器上按时间戳备份将被覆盖的精确文件；
3. 把候选文件及完整的 `require()` 依赖一起上传到临时目录，先逐个执行 `node --check`；
4. 检查通过后才覆盖生产文件；共享模块一次只带一台服务重启验收；
5. 验证 `systemctl is-active`、本机 `/health`、公网入口和客户端行为；有鉴权的 `/mcp` 还要确认匿名请求返回 401；
6. 任一步失败就停止扩大变更，用刚建立的备份回退。

PowerShell 会吃掉远程命令里的引号

现象 | 在 PowerShell 中执行 `ssh host "复杂 bash 命令"`，里面的双引号经过 PowerShell、Windows 原生命令参数解析和远端 shell 多层处理后丢失。远端最终收到裸括号或残缺 JSON，常见报错是 `syntax error near unexpected token '('`。这不是 bash 脚本本身有问题。

复杂脚本不要继续堆转义。把完整脚本写进 PowerShell here-string，以 UTF-8 编成 base64，远端只接收由 ASCII 字符组成的短命令：

```
$remoteScript = @'
set -euo pipefail
node --check /tmp/mcp-candidate/server.js
'@
$encodedScript = [Convert]::ToBase64String(
  [Text.Encoding]::UTF8.GetBytes($remoteScript)
)
ssh my-vps "printf '%s' '$encodedScript' | base64 -d | bash"
```

需要给远端 `curl --data` 传 JSON 时同样先编码正文，再从标准输入交给 curl，避免让 JSON 穿过多层引号解析：

```
$requestJson = '{"jsonrpc":"2.0","id":1,"method":"tools/list"}'
$encodedJson = [Convert]::ToBase64String(
  [Text.Encoding]::UTF8.GetBytes($requestJson)
)
ssh my-vps "printf '%s' '$encodedJson' | base64 -d | curl --data-binary @-
http://127.0.0.1:PORT/mcp"
```

base64 只是编码，不是加密。token、口令等凭据不要写进脚本、命令行、仓库或日志；需要认证的探测应该从服务器上权限受控的配置文件读取，且输出不回显凭据。

第 七 节

排错速查

现象	先查这里
`Couldn't register with ... sign-in service`	authless 服务的三个探测端点是否返回 404 + JSON
公网 403、本机 200，body 是 JSON-RPC `Invalid Host`	`allowedHosts` 没配对外域名
该 401 的地方返回 500	AuthInfo 缺 `expiresAt`；错误不是 `OAuthError` 子类；错误消息有非 ASCII

现象	先查这里
该 400 的注册返回 500	`registerClient` 抛的不是 `InvalidClientMetadataError`
授权最后一步失败	redirect_uri 与注册时不完全一致
注册就报 `redirect_uris must be https URLs...`	那端的回跳不在受信名单里（http loopback、别的域名）
网页端连不上但服务本机正常	服务是否公网可达（网页端从平台机房连，不是从你的电脑）
本机客户端能连、网页端不能	同上；再查 OAuth 白名单里有没有那家的域名

参考

Authentication for connectors（Claude 官方：连接器认证要求）
claude.com/docs/connectors/building/authentication

Add a connector that isn't in the directory（Claude 官方：添加自定义连接器）
claude.com/docs/connectors/custom/remote-mcp

Apps SDK Authentication（OpenAI 官方：ChatGPT 连接 MCP 的认证要求）
developers.openai.com/apps-sdk/build/auth

Create custom MCP server（OpenAI 官方：ChatGPT 支持读写工具）
developers.openai.com/api/docs/guides/custom-mcp-server

Build an MCP server（OpenAI 官方：readOnlyHint 等工具声明）
developers.openai.com/plugins/build/mcp-server

MCP 授权规范
modelcontextprotocol.io/specification/2025-11-25/basic/authorization